

Toward Real-Time Sentence-Level Sign Language Translation

Thanh-Hoang Nguyen Doan

The University of Danang – University of Science and Technology

102230150@sv1.dut.udn.vn

Abstract

Most sign language understanding systems operate at the level of isolated signs, limiting their usefulness in natural communication. We study *sentence-level* sign language translation (SLT) with the primary goal of real-time deployment rather than proposing a new translation architecture. We fine-tune a SHuBERT-ByT5 translation stack on a uniformly sampled 9,872-example subset of How2Sign, selected because of compute and storage constraints, using QLoRA while keeping SHuBERT frozen. The model obtains a validation BLEU of 16.7 and, on the test split, BLEU 15.9 and BLEURT 44.7. The main contribution is a hardware-aware streaming system: a Raspberry Pi 4B reference client provides camera capture, local text display, and speech output, while compute-intensive perception and translation run on a CPU/GPU backend. The capture protocol remains client-agnostic, so the same backend can serve a browser, phone, or laptop. Chunked ingestion, bounded queues, parallelized perception, temporal reordering, and a sentence-boundary state machine reduce mean post-finalization response latency from 1.873 to 1.354 seconds (27.71%) and P95 latency from 2.919 to 2.130 seconds (27.03%) over the complete 9,872-example working subset.

1 Introduction

Sign languages are complete natural languages with their own lexicon, grammar, and non-manual grammatical markers. Computational work on sign languages has historically concentrated on *isolated sign language recognition* (ISLR), in which a short, trimmed clip is mapped to a single gloss or word. This framing is convenient for data collection, annotation, training, and evaluation, because every example contains exactly one linguistic unit, allowing a model to focus on discriminative visual cues such as hand shape, hand location, movement direction, facial expression, and body posture. However, a single sign is a coarse unit of meaning: in isolation it often fails to carry the speaker’s full communicative intent.

In real interaction, signers do not produce discrete words separated by clean boundaries. They produce continuous streams of signs in which movements are chained together, co-articulated, and modulated in rhythm. For this reason we shift the target from word-level recognition to *sentence-level* sign language translation (SLT), where the input is an entire continuous utterance and the output is a complete text sentence (Camgöz et al., 2018, 2020). Operating at the sentence level lets the model learn contextual relations between signs, resolve ambiguities that arise when a sign appears alone, and produce output that can be shown as text or spoken directly, rather than a sequence of gloss labels.

This shift comes at a cost. Sentence-level SLT must process longer video with more frames, contend with fuzzy

boundaries between signs, and rely on richer supervision. Non-manual channels—facial expression, mouthing, and upper-body motion—become obligatory signals rather than optional cues; a model that attends only to the hands discards a substantial part of the linguistic information.

Deployment adds a further constraint. A useful assistive system should respond quickly, yet translating each sign the instant it appears is both error-prone and unnatural. We therefore adopt an *utterance-then-translate* design: the system continuously ingests a signing stream, detects when an utterance ends, and emits a translation shortly afterwards. The engineering goal is to balance the contextual accuracy of a sentence-level model against a response latency low enough for near real-time communication.

To realize this design we build on SHuBERT (Gueuwou et al., 2025), a self-supervised model that learns sign representations from roughly a thousand hours of sign-language video. We do not train a new sign encoder from scratch. Instead, we attach a byte-level decoder and fine-tune the translation stack on a resource-constrained, uniformly sampled subset of the How2Sign benchmark (Duarte et al., 2021), updating only a small set of parameters while SHuBERT remains frozen. The resulting model is computationally heavy, which is precisely why the core of this work is a hardware-aware real-time stack. Our reference implementation uses a Raspberry Pi 4B with a camera, display, speaker, battery, and enclosure as the interaction endpoint, while CPU/GPU inference is offloaded to a backend. The network contract is client-agnostic: a browser, phone, or laptop can replace the Raspberry Pi as long as it samples, chunks, and uploads frames through the same interface.

Contributions. The primary focus of this paper is real-time deployment, and our contributions reflect that emphasis.¹ (i) *Real-time streaming inference (main contribution)*. We present a streaming pipeline for continuous signing that combines chunked ingestion, bounded queues, parallelized landmark perception, temporal reordering, and a sentence-boundary state machine. With the model and decoding settings held fixed, the optimized runtime reduces mean post-finalization response latency by 27.71% and P95 by 27.03%. (ii) *Hardware-aware edge deployment*. We realize the interaction loop on a portable Raspberry Pi 4B appliance with camera capture, local display, and speech output, while offloading heavy inference to a GPU backend. The hardware prototype is a reference client, not a protocol dependency; browser, phone, and laptop clients use the same HTTPS in-

¹Code is available at <https://github.com/NguyenDoanThanhHoang/Toward-Real-Time-Sentence-Level-Sign-Language-Translation>.

terface. (iii) *How2Sign fine-tuning*. We fine-tune a frozen SHuBERT encoder with a ByT5 decoder on a uniformly sampled 9,872-example How2Sign subset using QLoRA, obtaining validation BLEU 16.7 and test BLEU 15.9, together with a test BLEURT score of 44.7.

2 Related Work

From isolated recognition to translation. Early sign-language systems targeted ISLR, mapping trimmed clips to glosses. Generic skeleton-based video encoders such as PoseConv3D (Duan et al., 2022) provide motion representations, while NLA-SLR (Zuo et al., 2023) specifically targets isolated sign recognition; neither method directly generates fluent sentences. Neural SLT reframes the problem as sequence-to-sequence generation: Camgöz et al. (2018) introduced neural sign language translation, and Camgöz et al. (2020) jointly modeled recognition and translation with transformers. Yin and Read’s STMC-Transformer (Yin and Read, 2020) strengthened translation sequence modeling and highlighted limitations of gloss-based representations. Later work removed the dependence on gloss supervision through visual–language pretraining and gloss-free end-to-end training (Zhou et al., 2023; Lin et al., 2023). Community benchmarks such as WMT-SLT (Müller et al., 2022) have further standardized evaluation. Our system follows the gloss-free, translation-oriented line: it maps continuous signing directly to sentences without an intermediate gloss decoding step, avoiding error propagation from a gloss recognizer into the language model.

Self-supervised representations. Self-supervised pretraining has reshaped speech and vision. HuBERT (Hsu et al., 2021) learns speech representations by masked prediction over clustered targets. SHuBERT (Gueuwou et al., 2025) carries this idea to sign language: it is a self-supervised transformer encoder trained on roughly 1,000 hours of American Sign Language video that predicts clustered targets over multiple visual streams—hands, face, and body pose—and reaches state-of-the-art results across sign language translation, isolated sign recognition, and fingerspelling detection. SHuBERT is the backbone of our system: we take its pre-trained multi-stream encoder as a frozen temporal sign representation and adapt only a byte-level decoder on top of it for sentence-level translation (Section 3). On the vision side, DINOv2 (Oquab et al., 2024) produces robust general-purpose image features from Vision Transformers (Dosovitskiy et al., 2021) without labels; we use it as the frozen regional feature extractor that supplies SHuBERT’s per-frame inputs.

Text generation and efficient adaptation. T5 (Raffel et al., 2020) casts NLP tasks as text-to-text generation; ByT5 (Xue et al., 2022) removes the fixed subword vocabulary by operating on bytes, which suits short, free-form target sentences. Because full fine-tuning of such models is expensive, we adopt parameter-efficient adaptation: LoRA (Hu et al., 2022) injects low-rank updates into attention projections, and QLoRA (Dettmers et al., 2023) combines this with 4-bit quantization so that adaptation runs on modest hardware.

Perception and evaluation. For per-frame geometry we rely on MediaPipe (Lugaresi et al., 2019) and its pose estimator (Bazarevsky et al., 2020). For translation quality we report BLEU (Papineni et al., 2002) and BLEURT (Sellam et al., 2020).

3 Translation Model

Our model does not translate raw pixels directly. Instead it first converts video into structured visual signals—hand and face geometry, upper-body pose, cropped hand and face regions, and their evolution over time—and then passes those signals through three learned stages. Figure 1 summarizes the cascade. We describe each stage in turn and clarify which components are learned during adaptation and which are frozen.

3.1 Geometric perception

The first stage is a visual perception layer that estimates the structure of the signer in every frame. We use MediaPipe (Lugaresi et al., 2019) with three branches: a face landmarker, a hand landmarker for both hands, and a pose estimator for the upper body. These branches are not translation models; they produce geometric coordinates that tell the system *where* to look before any deep encoder runs.

The three branches play complementary linguistic roles. The hands are the primary carrier of lexical content. The face supplies expression, mouthing, and other non-manual markers that can flip a statement into a question or change sentiment. The pose stabilizes the interpretation of movement direction and helps detect whether the signer is active. Because these channels carry distinct information, none can be dropped without risking a loss of meaning, particularly in sentences where expression or trajectory is grammatically load-bearing.

3.2 Regional visual features

Rather than feeding whole frames to the temporal encoder, we use the landmarks to localize the hands and face, crop those regions, and resize them to 224×224 . Each region is encoded by DINOv2 (Oquab et al., 2024), a self-supervised Vision Transformer, into a 384-dimensional feature vector that compresses hand shape, finger orientation, relative hand position, and facial expression. This has two benefits. First, discarding the background reduces environmental noise. Second, DINOv2 features are more stable than raw pixels, letting the downstream translator focus on the semantic evolution of a sign rather than re-solving low-level vision.

3.3 Temporal sign encoding

Given per-frame features, the model must understand how they change over time. This is the role of SHuBERT (Gueuwou et al., 2025), a sequence encoder for sign language inspired by HuBERT (Hsu et al., 2021). SHuBERT consumes four channels—left hand, right hand, face, and upper-body pose. Hands and face are represented by their 384-d DINOv2 vectors, while pose is reduced to a low-dimensional geometric vector derived from nose, shoulder, elbow, and

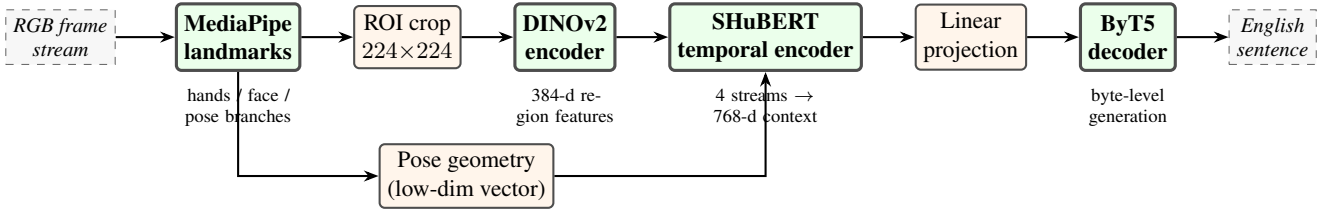


Figure 1: The sentence-level translation cascade. Landmarks locate the hands, face, and upper body; hand and face crops are encoded by DINOv2 into 384-d region features, while the pose branch contributes a low-dimensional geometric vector. SHuBERT fuses the four streams into a 768-d contextual sequence, and a linear projection feeds ByT5, which decodes an English sentence. Only the projection and selected ByT5 parameters are trained; SHuBERT is frozen.

wrist points. The channels are projected into a common space, fused per time step, and encoded by a transformer (Vaswani et al., 2017) into a 768-dimensional contextual representation of the whole utterance. Because this representation integrates past and future frames within the same sentence, it disambiguates signs that share a hand shape but differ in trajectory or context.

3.4 Text generation

The final stage is ByT5 (Xue et al., 2022), an encoder-decoder text generator. After SHuBERT produces the contextual representation, a projection layer maps it into the decoder’s latent space, and ByT5 generates the English sentence step by step. Operating at the byte level frees the output from a fixed subword vocabulary, which is convenient for the short, free-form target sentences in our data. During real-time inference we use a small beam width to keep decoding fast; in preliminary runs, larger beams did not improve short sentences and sometimes added trailing artifacts.

3.5 Parameter-efficient adaptation

We initialize the translation stack from pretrained SHuBERT and ByT5, then fine-tune it for SLT on the uniformly sampled How2Sign working subset (Duarte et al., 2021) with QLoRA (Dettmers et al., 2023). This is task adaptation on How2Sign rather than training the sign encoder from scratch. The base ByT5 weights are quantized to 4-bit NF4 with an FP16 compute type, and low-rank LoRA (Hu et al., 2022) updates ($r = 4$) are trained on the query and value projections together with the SHuBERT→ByT5 projection layer. SHuBERT itself is frozen. This choice preserves the self-supervised sign prior, restricts trainable parameters to components that directly affect generation, and makes fine-tuning affordable: the whole run fits on two NVIDIA T4 GPUs (Section 5).

4 Real-Time System and Deployment

The learned model is accurate but heavy, so practical responsiveness depends on hardware–software co-design. We separate three concerns: (1) a capture and interaction client, (2) a CPU runtime for ingestion, perception, ordering, and boundary detection, and (3) a GPU translation stage. This separation lets the Raspberry Pi prototype provide a complete assistive interface without requiring the full model to execute

Component	Role in the reference client
Raspberry Pi 4B	Coordinates capture, display, audio, and networking.
USB HD webcam	Captures the continuous signing stream.
On-device display	Shows the translated sentence.
Speaker + amplifier	Speaks the translation aloud.
Battery, converter, fan	Provides portable power and thermal stability.
3D-printed enclosure	Packages the components as a hand-held appliance.

Table 1: Reference edge hardware. The client handles capture and user interaction; compute-intensive models are off-loaded to the backend.

on-device, while preserving the same backend contract for other client types. Figure 2 shows the resulting runtime.

4.1 Hardware prototype and client abstraction

The reference deployment is a portable Raspberry Pi 4 Model B appliance that implements the complete user-facing loop: a USB webcam captures the signer, an on-device screen displays the translated sentence, and a small speaker driven by a Class-D amplifier reads it aloud. A battery pack, step-down converter, cooling fan, and 3D-printed enclosure make the unit portable and thermally stable (Table 1). The Raspberry Pi coordinates capture, local I/O, and networking; CPU/GPU-intensive perception and translation are executed on a backend reached over Wi-Fi/HTTPS.

The hardware is deliberately separated from the client protocol. The backend requires only that a client sample frames, encode them, group them into chunks, and upload them over HTTPS. The Raspberry Pi is therefore a concrete deployment and evaluation client, while a browser, phone, or laptop can replace it without changing the model or server-side streaming logic. Throughout the remainder of the paper, *client* denotes this generic interface and *Raspberry Pi prototype* denotes the physical reference implementation.

4.2 Chunked ingestion

The client samples the incoming video below the native camera rate to reduce computation while preserving motion, normalizes each frame to a fixed size, compresses it to JPEG, and groups a small number of frames into a chunk. Chunking

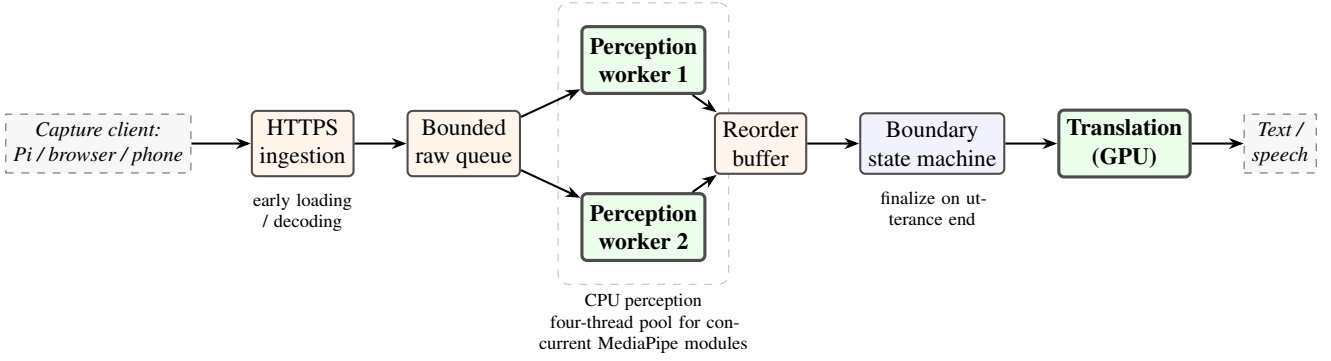


Figure 2: The streaming inference pipeline. The reference capture client is a Raspberry Pi appliance, but the interface also accepts browser, phone, or laptop clients. Frames are sampled, chunked, and sent over HTTPS. A bounded queue feeds two CPU perception workers whose MediaPipe modules execute concurrently; a reorder buffer restores temporal order; a boundary state machine finalizes the utterance; and the GPU translation stage emits text, which the client may also speak aloud.

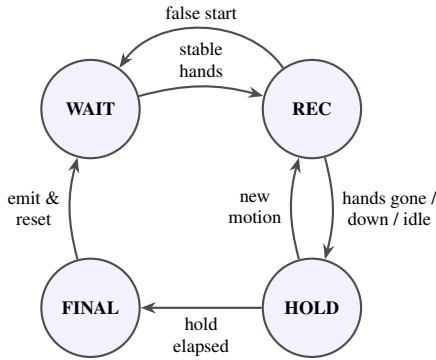


Figure 3: Sentence-boundary detection as a finite-state machine. Transitions are governed by motion thresholds and timing windows; the HOLD state prevents duplicate finalizations from a single ending gesture.

amortizes network cost relative to sending single frames, yet the chunks remain short enough for near real-time response. Concretely, the client samples at 15 fps, normalizes to 640×480 at JPEG quality 72, and groups ten consecutive frames per chunk (Section 5).

4.3 Sentence-boundary detection

Because we translate whole utterances, the system must decide when an utterance begins and ends. A boundary detector runs as a state machine (Figure 3). In the WAITING state no stable signing is observed. When hands and motion become sufficiently stable, it moves to RECORDING and accumulates features. A brief, unstable appearance is treated as a false start and returns the machine to WAITING. When the hands disappear, drop, or become nearly still beyond a threshold, the machine enters a short HOLD to avoid emitting duplicate sentences from a single ending gesture, and finally FINALIZED, at which point the model generates the sentence. When the user stops manually, the system still drains already-received chunks before closing the session.

Optimization	Effect
Early loading	Lowers per-chunk start-up delay.
Bounded queue	Keeps latency stable under bursty arrival.
Chunk coalescing	Reduces tail latency while limiting information loss.
Two perception workers	Increases CPU throughput while bounded queues limit backlog.
Concurrent MediaPipe modules	Perception drops from 107–111 to 45–48 ms/frame.
Reorder buffer	Preserves temporal correctness under parallelism.
Safe drain	Avoids losing end-of-utterance chunks.

Table 2: Runtime optimizations and their effect.

4.4 Temporal synchronization

Signing is sequential, so frame order matters as much as frame content. With several workers running in parallel, chunks may finish out of order. A reorder buffer holds completed chunks and releases them to the translator only when all earlier chunks are ready, so the system exploits parallelism while preserving the temporal logic of the utterance.

4.5 Latency optimizations

Table 2 lists the optimizations. *Early loading* decodes and enqueues data as soon as enough sub-frames arrive, instead of waiting for a full request. A *bounded queue* caps depth to keep latency stable when the arrival rate exceeds instantaneous throughput; under overload the system *coalesces* incoming data into the most recent chunk rather than letting a backlog grow. *Two perception workers* increase CPU throughput, and within each worker the face, hand, and pose modules execute *concurrently* through a four-thread pool, so wall-clock perception time is governed by the slowest module rather than the sum of all three. The *reorder buffer* preserves correctness, and a *safe-drain* rule finishes in-flight chunks before producing the final output so that no end-of-utterance content is lost.

Property	Value
Total examples	9,872
Unique target sentences	9,746
Signers	9
Avg. examples per sentence	1.013
Train split	7,000
Validation split	1,121
Test split	1,751

Table 3: Resource-constrained, uniformly sampled How2Sign working subset: statistics and data splits.

Component	Setting
Training hardware	$2 \times$ NVIDIA T4
Quantization	QLoRA 4-bit (NF4), FP16 compute
LoRA rank	$r = 4$
Trainable	ByT5 q, v + projection layer
Frozen	SHuBERT
Effective batch	$16 \times 16 \times 2 = 512$

Table 4: Final fine-tuning configuration.

5 Experimental Setup

5.1 Data

How2Sign (Duarte et al., 2021) is a large-scale continuous American Sign Language corpus of instructional videos in which each example pairs a signing clip with an English sentence. The original corpus contains substantially more data than we could process within the available compute and storage budget. We therefore extract a uniformly sampled working subset of 9,872 clip–sentence pairs from the available corpus. The extracted subset contains 9 signers and is split into 7,000 training, 1,121 validation, and 1,751 test examples (Table 3). The subset retains the sentence-level clip–text task while fitting the available resource budget. Of the 9,872 target sentences, 9,746 are unique (≈ 1.013 examples per sentence), indicating low exact-sentence repetition. We report this as a dataset statistic rather than as evidence of an open-vocabulary setting.

5.2 Training configuration

Fine-tuning on the sampled How2Sign working subset follows the QLoRA recipe of Section 3.5. Table 4 lists the configuration. Training runs on two NVIDIA T4 GPUs with an effective batch size of 512, formed from a per-device batch of 16 accumulated over 16 steps across 2 devices. The base ByT5 is loaded in 4-bit NF4 with FP16 compute, LoRA rank is $r = 4$, and only the ByT5 query/value projections and the sign→text projection are trainable; SHuBERT is frozen.

5.3 Runtime deployment configuration

The reference latency path uses the Raspberry Pi 4B client described in Section 4.1. It samples video at 15 fps, resizes frames to 640×480 , encodes JPEG at quality 72, and sends chunks over Wi-Fi/HTTPS to the backend. The optimized setting uses ten frames per chunk and five concurrent

Parameter	Value	Meaning
Min. utterance	600 ms	discard false starts
Hands absent	400 ms	finalize after disappearance
Hands lowered	500 ms	finalize after lowering
Hands idle	900 ms	finalize after sustained stillness
Hand motion	0.004	below threshold is jitter/stillness
Body motion	0.018	suppress natural postural sway

Table 5: Sentence-boundary detection parameters used at inference.

requests. These measurements therefore include the reference hardware/client path, while the server API remains unchanged for browser, phone, or laptop clients.

5.4 Metrics

Quality. We report BLEU (Papineni et al., 2002) and BLEURT (Sellam et al., 2020). BLEU measures n -gram overlap and applies a brevity penalty, while BLEURT is a learned BERT-based metric intended to capture semantic and fluency similarities that lexical overlap can miss. BLEURT is reported for the held-out test split.

Latency. For real-time behavior we measure the delay between the moment the system finalizes an utterance and the moment it emits text. For evaluated clip i ,

$$L_i = t_{\text{output},i} - t_{\text{finalize},i}, \quad (1)$$

and the mean latency over N clips is $\frac{1}{N} \sum_i L_i$. We also report the 95th percentile (P95). For an aggregate latency statistic M (mean or P95), the relative reduction is

$$\text{Reduction}_M(\%) = \frac{M_{\text{base}} - M_{\text{opt}}}{M_{\text{base}}} \times 100. \quad (2)$$

Boundary-detection parameters. The real-time evaluation uses the state-specific timing and normalized-motion thresholds in Table 5. Separate windows for absent, lowered, and idle hands avoid forcing all end-of-utterance conditions into a single timeout.

6 Results and Analysis

6.1 Translation quality

Table 6 reports the final quality after fine-tuning on the sampled How2Sign working subset. BLEU is 16.7 on validation and 15.9 on test; the held-out test split additionally obtains BLEURT 44.7. The validation and test BLEU scores differ by 0.8 points. BLEURT provides a complementary semantic measure beyond surface n -gram overlap.

Split	BLEU	BLEURT
Validation	16.7	–
Test	15.9	44.7

Table 6: Final translation quality after fine-tuning on the sampled How2Sign working subset.

Factor	Baseline	Optimized
Data loading	multipart	early loading
Workers	1	2
MediaPipe execution	sequential	concurrent (4-thread pool)
Sampling rate	15 fps	15 fps
Frames / chunk	15	10
Concurrency	1	5
Final beam	2	2
Image / JPEG	640×480, 72%	640×480, 72%

Table 7: Baseline vs. optimized runtime configuration. Model and input quality are held fixed.

6.2 Real-time latency

We compare a sequential *baseline* configuration with the *optimized* configuration; the two differ only in runtime settings (Table 7) and share the same translation model, image quality, sampling rate, frame size, and final beam width, so any difference is attributable to the streaming stack rather than the model.

Runtime evaluation covers every example in the complete 9,872-example working subset. The optimized system reduces mean latency from 1.873 s to 1.354 s (a 27.71% reduction) and P95 latency from 2.919 s to 2.130 s (27.03%); see Table 8 and Figure 4. The aggregate results therefore reflect the full sampled working corpus rather than a small sentence-level evaluation slice. This is a systems workload measurement; it should not be interpreted as an estimate of translation generalization on the training portion of the subset.

6.3 Where the gains come from

The dominant factor is perception time. In the sequential baseline, each frame waits for the face, hand, and pose branches in turn, so per-frame cost approximates the sum of the branches (about 107–111 ms). Running the MediaPipe modules concurrently through a four-thread pool makes wall-clock perception time track the slowest module instead, dropping it to about 45–48 ms/frame. Early loading and concurrent requests further shorten the client–server wait by letting the pipeline begin before a full request has arrived. Throughout these runs the raw and prepared queues never exceeded a depth of one and never saturated, confirming that the bottleneck was perception latency rather than queueing.

6.4 Qualitative examples

Table 9 shows representative sentences with their baseline and optimized latencies. The optimized system is consistently faster across short and long sentences, and longer

Metric	Baseline	Optimized	Reduction
Mean latency	1.873 s	1.354 s	27.71%
P95 latency	2.919 s	2.130 s	27.03%

Table 8: Post-finalization latency over the complete 9,872-example How2Sign working subset.

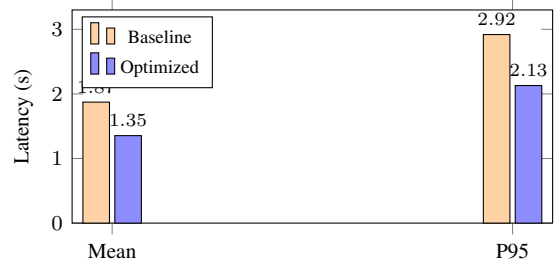


Figure 4: Mean and P95 latency, baseline vs. optimized.

utterances retain proportionally larger absolute savings, consistent with per-frame perception being the main cost.

6.5 Recommended defaults

From these experiments we recommend, as project defaults: two perception workers; a four-thread intra-worker pool for MediaPipe; ten frames per chunk; five concurrent client requests; a final beam of two; and state-specific finalization windows of 400/500/900 ms for hands disappearing, lowering, or remaining idle. This configuration keeps the translation model and decoding settings fixed while materially reducing latency and remaining simple enough to reason about as a real-time system.

7 Conclusion

We presented a hardware-aware, sentence-level sign language translation system whose main objective is responsive deployment. The translation stack is fine-tuned with QLoRA on a uniformly sampled 9,872-example How2Sign subset while SHuBERT remains frozen, reaching BLEU 16.7 on validation and, on test, BLEU 15.9 and BLEURT 44.7. The central contribution is the runtime and deployment design: a portable Raspberry Pi 4B interaction client paired with a client-agnostic HTTPS interface and an offloaded CPU/GPU backend. Chunked ingestion, bounded queues, concurrent perception, temporal reordering, and sentence-boundary detection reduce mean post-finalization response latency from 1.873 to 1.354 seconds (27.71%) and P95 from 2.919 to 2.130 seconds (27.03%). The hardware prototype demonstrates the complete camera-to-text-and-speech loop, while the generic client contract allows the same backend to support browser, phone, or laptop capture.

Limitations

Several limitations remain. First, the system responds after an utterance ends rather than translating incrementally while the user is still signing; truly incremental decoding is

Sentence	Base	Opt
Hi!	1.163	0.846
And ukuleles are different.	1.138	0.828
And this is the base plate.	1.441	1.027
All telescopes should come with a finder.	1.428	1.047
Everything fused really nice.	1.356	0.972
And that's how you tune a ukulele.	1.242	0.894
Again, one more time we'll show it for you.	1.996	1.433
Each has a unique feel, and there's no one particular one that's right for everyone.	3.044	2.175

Table 9: Representative sentences and their latency (s) under the baseline and optimized configurations.

future work. Second, post-finalization latency is measured over the complete 9,872-example working subset under controlled deployment conditions; absolute numbers may shift under network variability or heavier concurrent load. Third, although the client protocol is generic, the reported hardware-backed latency measurements use the Raspberry Pi reference client; browser, phone, and laptop clients require separate benchmarking. Finally, the multi-stage pipeline is computationally non-trivial, and pushing the full model to run entirely on the edge remains challenging.

Ethics Statement

The intended use of this work is accessibility: supporting communication for deaf and hard-of-hearing signers and non-signers. Because the input is video of a person, privacy is a first-order concern; the system de-identifies faces by graying and blurring while retaining the eyes and mouth, which are linguistically necessary for non-manual markers, thereby reducing identity exposure without discarding signal. Responsible deployment requires informed consent for data collection, attention to signer diversity so the model does not systematically fail for underrepresented signers or dialects, and clear communication that automatic translations can be wrong and should not be relied upon in high-stakes settings without human confirmation.

Acknowledgments

We thank Dr. Ninh Khánh Duy for his valuable feedback and guidance throughout this project.

References

- Valentin Bazarevsky, Ivan Grishchenko, Karthik Raveendran, Tyler Zhu, Fan Zhang, and Matthias Grundmann. BlazePose: On-device real-time body pose tracking, 2020. URL <https://arxiv.org/abs/2006.10204>.
- Necati Cihan Camgöz, Simon Hadfield, Oscar Koller, Hermann Ney, and Richard Bowden. Neural sign language translation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7784–7793, 2018. URL https://openaccess.thecvf.com/content_cvpr_2018/html/Camgoz_Neural_Sign_Language_CVPR_2018_paper.html.
- Necati Cihan Camgöz, Oscar Koller, Simon Hadfield, and Richard Bowden. Sign language transformers: Joint end-to-end sign language recognition and translation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10023–10033, 2020. URL https://openaccess.thecvf.com/content_CVPR_2020/html/Camgoz_Sign_Language_Transformers_Joint_End-to-End_Sign_Language_Recognition_and_Translation_CVPR_2020_paper.html.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, pages 10088–10115, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=YicbFdNTTy>.
- Haodong Duan, Yue Zhao, Kai Chen, Dahua Lin, and Bo Dai. Revisiting skeleton-based action recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2969–2978, 2022. URL https://openaccess.thecvf.com/content_CVPR2022/html/Duan_Revisiting_Skeleton-Based_Action_Recognition_CVPR_2022_paper.html.
- Amanda Duarte, Shruti Palaskar, Lucas Ventura, Deepti Ghadiyaram, Kenneth DeHaan, Florian Metze, Jordi Torres, and Xavier Giro-i Nieto. How2Sign: A large-scale multimodal dataset for continuous american sign language. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2735–2744, 2021. URL https://openaccess.thecvf.com/content_CVPR2021/html/Duarte_How2Sign_A_Large-Scale_Multimodal_Dataset_for_Continuous_American_Sign_Language_CVPR_2021_paper.html.
- Shester Gueuwou, Xiaodan Du, Greg Shakhnarovich, Karen Livescu, and Alexander H. Liu. SHuBERT: Self-supervised sign language representation learning via multi-stream cluster prediction. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28792–28810, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.1397. URL <https://aclanthology.org/2025.acl-long.1397/>.

- Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021. doi: 10.1109/TASLP.2021.3122291. URL <https://ieeexplore.ieee.org/document/9585401>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Kezhou Lin, Xiaohan Wang, Linchao Zhu, Ke Sun, Bang Zhang, and Yi Yang. Gloss-free end-to-end sign language translation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12904–12916, Toronto, Canada, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.722. URL <https://aclanthology.org/2023.acl-long.722/>.
- Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg, and Matthias Grundmann. MediaPipe: A framework for building perception pipelines, 2019. URL <https://arxiv.org/abs/1906.08172>.
- Mathias Müller, Sarah Ebling, Eleftherios Avramidis, Alessia Battisti, Michèle Berger, Richard Bowden, Annelies Braffort, Necati Cihan Camgöz, Cristina España-bonet, Roman Grundkiewicz, Zifan Jiang, Oscar Koller, Amit Moryossef, Regula Perrollaz, Sabine Reinhard, Annette Rios, Dimitar Shterionov, Sandra Sidler-miserez, and Katja Tissi. Findings of the first WMT shared task on sign language translation (WMT-SLT22). In *Proceedings of the Seventh Conference on Machine Translation*, pages 744–772, Abu Dhabi, United Arab Emirates (Hybrid), 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.wmt-1.71. URL <https://aclanthology.org/2022.wmt-1.71/>.
- Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=a68SUt6zFt>.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, 2002. Association for Computational Linguistics. doi: 10.3115/1073083.1073135. URL <https://aclanthology.org/P02-1040/>.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL <https://www.jmlr.org/papers/v21/20-074.html>.
- Thibault Sellam, Dipanjan Das, and Ankur Parikh. BLEURT: Learning robust metrics for text generation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7881–7892, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.704. URL <https://aclanthology.org/2020.acl-main.704/>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need>.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022. doi: 10.1162/tacl_a_00461. URL <https://aclanthology.org/2022.tacl-1.17/>.
- Kayo Yin and Jesse Read. Better sign language translation with STMC-transformer. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 5975–5989, Barcelona, Spain (Online), 2020. International Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.525. URL <https://aclanthology.org/2020.coling-main.525/>.
- Benjia Zhou, Zhigang Chen, Albert Clapés, Jun Wan, Yanyan Liang, Sergio Escalera, Zhen Lei, and Du Zhang. Gloss-free sign language translation: Improving from visual-language pretraining. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 20871–20881, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/html/Zhou_Gloss-Free_Sign_Language_Translation_Improving_from_Visual-Language_Pretraining_ICCV_2023_paper.html.
- Ronglai Zuo, Fangyun Wei, and Brian Mak. Natural language-assisted sign language recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14890–14900, 2023. URL https://openaccess.thecvf.com/content/CVPR2023/html/Zuo_Natural_Language-Assisted_Sign_Language_Recognition_CVPR_2023_paper.html.